

Next：以直接担保实现连续 UTXO 支付

Lu Hengrun

摘要—连续支付要求收款方在先前交易执行之前复用收到的价值。Next 在未花费交易输出 (UTXO) 账本中，通过注册的担保组织支持这一点。收款方验证一份经法定人数签名的输出证书 (TXCer)，并在公共执行异步进行的同时请求后继交易。若已认证的源交易缺失，委员会执行该后继交易，并记录一项由签发组织备付金支撑的义务。后继交易所在的区块公开源交易的证书，使原签署者能够依据保存的批准补交该支付。期限之后，一项公共赔付决定对仍处于 Open 状态的义务扣减备付金并将其关闭，无需门限适配；之后到达的源交易则偿还赔付它的签发组织。在固定成员与拜占庭故障的设定下，我们证明了唯一消费、所有证书未清偿 CAL 负债（包括未公开提交的证书）有资金支撑，以及延迟中性。受限的历史表示命令把备付金扣减记录进已存储的区块，并保持所有者授权、后继交易引用和完整区块身份不变。在一台 M4 Max 主机上，基线版本的 100 跳链到达最后一次认证接收的中位耗时为 158.846 ms，而每一跳都等待公共确认时为 65.204 s。在 decision-first 版本（增加了补交和独立的赔付决定）中，全部适配服务暂停时，赔付与偿还仍然完成。

关键词—UTXO 支付，支付担保，拜占庭容错，授权的历史表示。

I. 引言

在 UTXO 账本中，一笔支付花费的是先前某笔交易创建的输出。如果收款方必须等到该交易完成公共执行，支付链的每一跳都会增加一次确认延迟。设想 Alice 向 Bob 支付 100 CAL，Bob 再把收到的输出转给 Carol。要在公共执行之前转出这个输出，Bob 需要可验证的证据，证明其价值有资金支撑。如果 Bob 的支付先执行，账本就消费了一个 Alice 的交易尚未创建的输出，缺失的 100 CAL 需要一个明确的资金来源。无论 Alice 的交易先于 Bob 的到达、后于它到达，还是始终不到达，账务都必须保持正确。

基于证书的支付系统已经把收款方一侧的可用性与公共处理分离开来。FastPay 和 Zef 支持低延迟的认证支付，Sui Lutris 则针对不同类型的状态访问，把广播路径与共识路径结合起来 [1]–[3]。支付通道协议依靠通道关系及其流动性实现快速转账 [4], [5]。Next 处

理的是原生账本上已认证支付的资金支撑问题，特别是所消费输出的源交易尚未执行的情形。源交易尚未执行的已认证输出，应当能够支撑一笔已公共执行的后继交易，并且应当由一个已提供备付资金的明确责任方对缺失的源交易负责。用户花费指定了消费路由的自有输出，注册组织为未解决的源交易提供支撑，交易对手无需预先注资的通道路径。

这里存在四个相互耦合的问题。第一，证书在公共登记之前就已流通，因此从公开登记的缺口开始记账会漏掉已经存在的承诺，而签名额度必须覆盖每一笔已认证支付的未清偿 CAL 负债，包括从未公开提交的支付。第二，源交易可能在委员会已经为其缺失的价值赔付之后才到达。迟到的源交易不能把同一笔价值既付给垫付它的担保人，又付给已经花掉它的收款方，否则延迟本身就会转移资金。第三，客户端可能持有源交易的材料却从不提交。签发组织的成员应能在赔付到期前补交已有的源交易，而赔付无需等待历史适配服务。第四，源交易一旦被赔付，读取已存储历史的人应当能够看出已消费的输入由哪份备付金提供资金。记录这一事实必须保持所有者授权、输出标识符以及后继交易所引用的承诺不变。仅有稳定的交易哈希，既无法界定也无法授权这样的修改。

Next 固定支付权利，并把未解决的源交易划归其直接签发组织负责。输出证书 (TXCer) 把一笔支付的输出绑定到一个明确的签发组织。收款方在验证并原子地记录输出及其 TXCer 之后，通过该输出指定的消费组织请求后继交易。该请求携带其直接输入的证书。所有者授权、花费状态、费用和资源限额则另行检查。

如果某个已认证的输入在公共创建状态中仍然缺失，委员会就原子地消费它，记录针对签发组织的直接义务，并创建子输出。在义务处于 Open 状态期间，子输出即为最终输出。承载该子交易的区块同时承载创建了被消费输出的那笔支付的证书。批准过该支付的签发组织成员，根据自己保存的批准记录重建该支付并补交，无需新的签名。如果源交易执行，义务即告履行。

如果在期限之后赔付决定执行时源交易仍未出现，该决定就扣减签发组织的备付金，由它提供缺失的支撑，收款方不会得到第二次入账。该决定无需门限适配。之后到达的源交易在同一转换中偿还赔付它的签发组织，并且不为收款方创建第二个输出。当所有源交易最终都执行时，延迟只改变备付金的临时占用，本金的最终分配与无延迟时相同。

赔付决定本身就是一条关闭义务的公共记录。随后，历史表示命令在既有承诺之下，把已消费输入的资金引用替换为备付金引用。已存储的区块指明为该输入提供资金的备付金扣减，并且仍可对照其原始区块身份验证，因此读者能够在区块本身中核对指定历史输入的资金来源。钱包、同步和重放继续读取原始命令。我们把这一设计归纳为三项职责各异的贡献。

C1: 面向一轮续花的分层架构。我们把成员认证、收款方交付、证据复制和公共执行分离开来。成员在签名之前，原子地锁定输入并预留额度。一个并行认证轮产生一份 TXCer。INSTALL 在组织内复制完整材料，公共提交独立推进，二者均在后台进行。注册组织在组织内部和组织之间使用同一个证据接口，支付本金与担保人备付金保持分离。

C2: 有资金支撑且延迟中性的直接责任记账。我们定义原子的缺失输入转换及其关闭路径：由源交易履行、由公共赔付决定赔付、由迟到的源交易偿还。对于固定的输入与证书形态，请求的祖先证据不随转账深度增长。在固定成员的拜占庭模型下，我们证明唯一消费与资产守恒，并证明签名额度约束了证书（包括未公开提交的证书）的未清偿 CAL 负债。对于有限的、互不冲突且源交易全部执行的支付集合，到达与赔付的任意交错，最终得到的 CAL 持有和备付金余额都与无延迟执行相同。

C3: 证据驱动的源交易解决与保持身份的历史表示。我们让原签署者依据后继交易公开的证书补交源交易，并将赔付决定与门限适配分离。受限的历史表示命令没有经济效果，保持所有者授权、输出引用及完整区块身份（含分片集合头）不变。同一区块的命令共享受影响分片最终表示的适配，安装则直接推进到已提交的修订版本。对于固定的已决定槽位集合，最终区块体不依赖于命令如何分组。

我们在一台 M4 Max 主机上评估两个版本：续花与记账基线版本，以及增加了补交和独立赔付决定的 decision-first 版本。在基线版本上，100 跳链完成最后

一次认证接收的三轮用时中位数为 158.846 ms，而在各跳之间等待公共确认则需要 65.204 s。三次各含 30,000 笔支付的运行（含后台排空）完成速率为 1,750–1,887 笔/秒，一个九次运行的矩阵关闭 21,600 笔支付，并偿还全部 216 次赔付。在 decision-first 版本上，三次运行的功能测试显示，被扣留的源交易无需赔付即可执行，而收款方的接收约需 14 ms。在所有适配服务暂停的情况下，赔付与偿还仍然完成；六轮普通路径比较中，运行级接收 P50 和 P95 的范围相互重叠。第 III 节和第 IV 节定义该设计，第 V 节和第 VI 节展开其安全性论证与评估。

II. 背景与相关工作

Next 把三件事结合在一起：收款方可在公共执行之前复用的认证支付，对尚未执行的源交易由备付金支撑的责任，以及对已存储历史的授权编辑。我们沿两个问题与既有工作比较：谁为缺失的源交易负责，以及该责任被记录之后，哪些支付身份和区块身份得以保留。

A. 基于证书的支付与执行

FastPay 使用拜占庭一致性广播实现低延迟支付，并在其主账本部署中以预付资金支撑兑付 [1]。它的可兑付性论证允许在所有入账转账尚未在本地确认之前，就确认一份有效证书，条件是补齐发送方缺失的此前出账证书；兑付随后到达主资金池。Zef 把一次性账户与私密、可转让的币结合起来，普通花费使用当前的币证书和消费证据，而不是完整的祖先历史 [2]。

Sui Lutriss 把针对自有对象的广播执行与针对共享对象交互的共识配合使用 [3]。Stingray 通过拜占庭有界计数器和 FastUnlock 支持并发交易 [6]。它的重叠额度和回收机制是 Next 资源记账的近似先例，但其可交换的工作负载处理争用的方式，与每笔交易都消费其前驱输出的支付链不同。

Next 针对的正是这样的链。委员会在源交易执行之前公共执行一次已认证的消费，并针对直接签发组织登记一项有资金支撑的义务。签名额度预留全部输出金额，并在扣除未偿还赔付后恢复，所以它同时约束已登记的负债和由从未公开提交的证书所携带的承诺。后继交易的执行还会公开源交易的证书，使签发组织自己的成员能够依据其保存的批准补交该源交易。

B. 通道与带抵押的担保

Blitz 用 pay-or-revoke 取代通道支付中的两阶段承诺, Donner 发展出应对 Domino 攻击的虚拟通道 [4], [5]。CryptoMaze 提供带不可链接部分支付的原子多路径通道支付, 并避免共享边上重复的链下合约 [7]。这三者都通过通道安排运作。Next 运行在原生账本上, 具有注册的消费路由和组织备付金, 所以交易对手既不需要双边通道, 也不需要二者之间预先注资的路径。

Snappy 通过带抵押的批准保护接受快速链上支付的商户, 可复用的支付担保则把隐私与可在限定时间窗口内复用的客户抵押结合起来 [8], [9]。Next 则把用户的支付本金与额外的组织支撑分开。它的赔付关闭已执行子交易所缺失的源交易, 并且不向该子交易的收款方发放第二次本金入账。

HyperService 通过公共的 Network Status Blockchain 和 Insurance Smart Contract 裁决跨链执行违规, 并在经济上逆转已提交交易的影响 [10]。Next 在同一条原生 UTXO 账本内以不同的方式划定责任边界: 赔付结算的是签发组织的直接义务, 并保持已执行后继交易的本金效果不变。

C. 受控编辑与历史一致性

变色龙哈希允许在同一摘要之下对应经授权的替代消息, 并支撑了可编辑区块链的设计 [11]。SDR 研究去中心化的基于属性的编辑权限与动态委托, Escaping 把委员会选择与投票移到共识层之外, 以缩短删改的等待 [12], [13]。Reconstructing Chameleon Hash 提出了关于不可区分性和抗碰撞性的更强定义 [14]。Next 使用已有的适配原语; 应用兼容替换的权限来自一项已提交的赔付决定以及公共的支付状态检查。

Reparo 提供一个可公开验证的修复层, 并可在修复状态时重新执行受影响的交易 [15]。Next 预先固定支付授权和输出。一项公共赔付决定对赔付只记账一次, 历史表示命令则替换被允许的资金表示, 而不重新执行后继交易。原始命令 (含各项决定) 仍然是经济重放的历史。

RedactChain 协调同一区块内的并发编辑及其预处理 [16]。CDEdit 使用可复用的特权令牌, 减少多次编辑中令牌与密钥的重复生成与验证 [17]。在 Next 中, 一个批处理在同一区块的各已决定槽位之间共享最终区块表示。每个槽位的备付金扣减已由其决定提交过一次, 因此编辑本身没有经济效果。完整区块身份由稳定

的头部哈希和原始分片集合头共同构成, 公开提交的逻辑修订授权确切的替换区块体。因此, 物理安装可以直接推进到更晚的已授权修订, 而不改变后继引用或公共命令历史的重放。

III. 系统模型与概览

Next 让收款方在创建某个新认证输出的交易完成公共执行之前, 就能花费该输出。设计由四类对象承载: 经认证的支付权利、对这些权利的公共消费、对未解决源交易的责任, 以及该责任之后在已存储历史中的表示。表 I 列出了主要符号。

A. 参与者、资产与假设

Next 运行于原生 UTXO 账本之上。CAL 承载支付价值, FUEL 用于支付处理费用。用户花费自己的输出, 组织则另在公共备付金账户中持有 CAL。快速路径要求输出带有受支持的组织路由; 支付本金仍归其所有者, 不开设任何双边通道。费用默认由用户的最终 FUEL 输出支付, 同时也有一种经授权的组织出资模式。

每个组织有四个固定成员和一个可替换的收集者。成员独立验证请求并授权支付, 收集者则负责分发请求、汇集证书和协调传播, 不具有花费权限。另有一个由四名权重相等的验证者组成的委员会 $\mathcal{C}$, 需要三票通过, 它对公共命令排序, 并维护账户、输出、义务和赔付决定。

故障与密码学。在一次执行的全程中, 敌手静态地腐化每个组织至多一名成员以及委员会中的一名验证者; 用户和收集者的行为可以是任意的。诚实参与者共享网络配置、创世状态、规则和组织注册表, 每条路由指明一个有效配置和一个消费锁域。签名不可伪造, 规范摘要和原始区块承诺具有绑定性, 消息可以被任意延迟。

表 I
主要符号。

符号	含义
$G, \mathcal{C}$	担保组织与公共委员会
T, c_T	支付及其经认证的摘要事实
(x, i)	输出标识符与花费实例
g, B_g	授权额度 (grant) 与累计授权
a_T, p_T, Rec_T	CAL 输出总额、总赔付额与已偿还赔付额
$\mathcal{O}_x, \mathcal{D}_x$	缺失输出的直接义务及其不可变的赔付决定
(H_b, P_b)	稳定区块哈希与原始分片集合头
ρ_ℓ, ρ_p	已提交的逻辑修订与已安装的物理修订

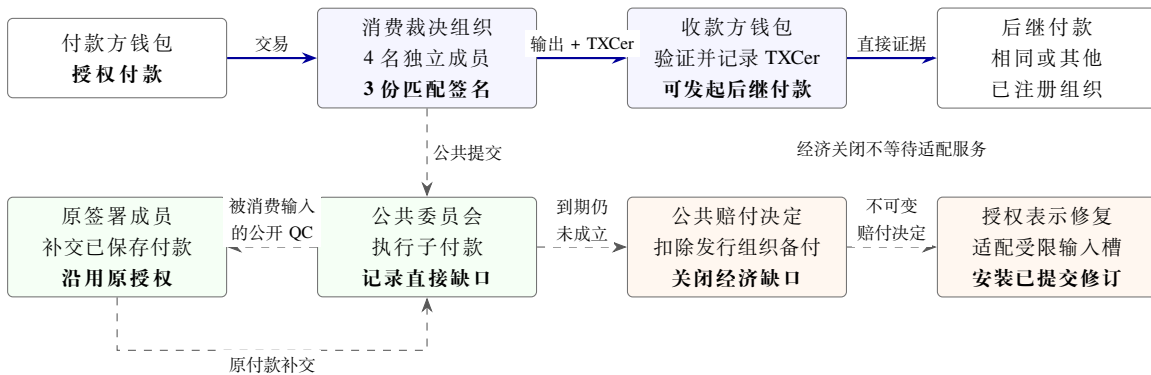


图 1. 单轮续花与公开证据驱动的来源处理。实线表示收款人路径，虚线表示后台工作。后继公开证据使原签署成员能够补交已保存的授权材料。来源到期仍未成立时，委员会独立完成赔付，不依赖适配服务。后续授权表示修复保持付款身份和完整区块身份。

状态。诚实成员的签名状态是原子的，既不会被遗忘，也不会回滚。成员签名之前，其批准记录请求的支付、准入向量和直接输入证书；法定人数证书不属于批准记录。公共执行是确定性的并以原子方式提交，跟随者应用经认证的连续区块，同时更新状态和游标。赔付与历史表示要读取原始区块及其元数据，因此正确的委员会副本在每个含有未关闭义务或等待历史表示的已决定槽位的高度上，都保留二者。

适配与进展。历史表示使用门限适配，其材料由可信的离线分发者生成并分发，任何适配输出都必须由一条已提交的命令授权。进展需要诚实参与者及时响应、证据可得、费用和资源充足，以及公平的投递与调度。实验所用的内存配置仅在进程运行期间保持状态原子，且每次运行都从全新的创世状态开始。

B. 输出、路由与证书

一个输出带有资产、金额、所有者和消费路由。其标识符 x 由网络、创建它的交易和输出位置推导而来，花费以 (x, i) 为索引，普通输出的 $i = 0$ 。快速支付的所有输入共享同一个消费组织，并由所有者授权该交易。新输出可以选择另一个注册组织，因此跨组织续花改变的是新输出的路由，而先前签发组织的责任保持不变。

TXCer 把一份摘要与一份法定人数证书 (QC) 配成一对：QC 是三名不同成员对同一事实 c_T 的签名。摘要绑定网络、交易身份、签发组织、配置、epoch、规则、输出承诺与金额，以及准入授权额度。输出体可以单独交付，收款方在保存输出之前检查其承诺、位置和所有权。该证书认证签发组织的担保，它既不是资产，也不是已登记的公共义务。

有三类事件反复出现：认证接收，即钱包在本地验证并记录一个输出；公共执行，即一条已提交的命令消费输入并创建输出；以及负债关闭，即一个缺失的源交易被履行或被赔付。赔付在经济上关闭一项负债，历史表示则随后在已存储历史中记录这一关闭。接收为后继请求提供证据，该请求还要经过自己的授权和资源检查。全文中，输出的源交易（也称父交易）指创建该输出的支付；其消费方（也称后继交易或子交易）指花费该输出的支付。子输出指消费方创建的输出。源交易到达指源交易的公共执行成功，实现中的 Settled 标志标记这一状态，它可以与非零的 Pending 并存。

C. 三层设计

认证与续花。成员在签名之前，原子地预留输入实例和相应资源。一份 CAL 授权额度 (grant) g 授权 B_g ，每个成员持有 $\lfloor 2B_g/3 \rfloor$ 的签名额度，并将其划分给本地的各个执行 Worker。一笔支付预留 a_T ，即其所有 CAL 输出（含找零）之和；重叠的额度约束承诺，不增加实际备付资金。收集者在一次并行的请求-响应轮中收到三份一致的批准后返回 TXCer。收款方验证并原子地本地记录即完成快速路径，而中继材料的存储、成员证据复制 (INSTALL) 和公共提交则在后台运行。输入锁的存续时间长于预算释放。

与源交易解耦的公共执行。一条公共命令携带交易、准入向量、组织授权和直接输入证书。委员会重建经认证的摘要，验证授权，并检查当前的消费状态。对于尚未公开创建的已认证输入，它原子地记录消费、针对签发组织登记整份证书的覆盖、创建 O_x 及其缺口，并创建子输出；只有实际被消费的缺失输出才会开立义务。每次公共支付检查都使用直接证书，而不是去获取祖先交

易体。成员依据经认证的区块，对照自己记录的扣减来恢复签名额度（第 IV-C 节）。创世时检查每个支撑账户下授权额度之和；充值扣减非保护账户，并原子地提高余额和授权额度；授权永不下降。

补交、赔付与历史表示。义务通过源交易到达而关闭；或者在由公共区块时间确定的期限之后，由签发组织的备付金赔付而关闭。执行消费方的区块会公开其源交易的证书，持有批准记录的签发组织成员根据保留的材料重建该支付并补交，无需新的签名。期限一过，公共赔付决定即可关闭仍处于 Open 状态的义务。该决定扣减备付金、关闭缺口，并记录一份不含适配数据的不可变决定；之后到达的源交易在其自身执行中偿还赔付它的签发组织，该决定仍保持已记录。

历史表示命令随后把已决定输入的资金引用替换为备付金引用。受限的变色龙哈希适配保持固定的授权、输出引用和完整区块身份 (H_b, P_b)；RepairBatch 把同一历史区块的已决定槽位归为一组，这些命令只写入历史表示状态。原始命令、已提交的修订 ρ_ℓ 和已安装的修订 ρ_p 保持分离，安装可以直接推进到更晚的已授权版本，重放运行原始命令，其中包含赔付决定。后继交易既不需要新的签名，也不需要重新执行。

D. 连续支付示例与设计目标

Alice 通过 G_A 向 Bob 支付 100 CAL，创建了路由到 G_B 的输出 x 。Bob 验证其 TXCer，并获得一份向 Carol 付款的支付的证书。如果 Bob 的支付先执行，委员会消费 $(x, 0)$ ，记录针对 G_A 的 100-CAL 义务，并创建 Bob 的输出。Carol 无需 Alice 的交易体即可从 Bob 的支付继续，而承载 Bob 支付的区块同时承载 Alice 支付的证书，所以当 Alice 的钱包不补交时，批准过该支付的 G_A 成员可以补交。

随之有三种结果。如果 Alice 的支付在赔付之前执行，它就关闭该义务，实例 0 仍保持已消费。期限一过，赔付决定即可关闭仍处于 Open 状态的义务，办法是从 G_A 扣减 100 CAL；Carol 不会得到第二次本金入账，Bob 的支付不会重新执行，已存储的区块保持原始资金引用，直到某条历史表示命令把它替换为备付金引用。如果 Alice 的支付在赔付之后执行，它按相同规则消费自己的输入，并向 G_A 偿还 100 CAL；该决定仍保持已记录，Bob 不会得到第二个输出， $(x, 0)$ 仍保持已消费。如果 Alice 的支付始终不执行，则由 G_A 承担损失。

Next 的目标是：在公共执行之前续花、有资金支撑的直接义务、基于公共证据补交源交易，以及在源交易解决前后保持后继交易不变。经济关闭通过委员会的公共决定完成，历史表示在快速路径之外随后进行。

IV. 协议设计

Next 认证固定的支付授权和输出，并允许它们的源交易保持未解决。公共执行消费一个缺失的已认证输入，并原子地记录签发组织的直接义务，同时公开源交易的证书，使原签署者能够补交该源交易。期限届满时仍然缺失的源交易，通过一项无需门限适配的公共赔付决定关闭，历史表示命令随后在既有承诺之下替换资金引用。

A. 身份、认证与交付

设 K_T 为编码后的支付体与输入声明， C_T 为输入承诺， Σ_T 为输出摘要：

$$\begin{aligned} K_T &= \text{Encode}(T.\text{Body}, T.\text{Claims}), \\ \text{id}(T) &= H(\text{TX_V3}, K_T, C_T), \\ c_T &= H(\text{DIRECT_CERT_V3}, \text{Encode}(\Sigma_T)). \end{aligned} \quad (1)$$

所有者签名认证 $\text{id}(T)$ ，并且不属于其计算范围；每个输出标识符绑定这一身份、网络和输出位置。资金引用与 opening 单独编码，因此受限的替换不会改变已授权的声明和输出。初次提交要求每个被声明的输入具有 OriginalFunding 及有效的 opening，之后的历史表示保持支付体、声明、承诺和所有者签名不变。因此验证缓存以完整的命令字节为键。

成员检查所有权、路由、直接输入证书、价值守恒和准入向量。一个本地事务重新检查输入锁、最终费用输入以及指定 Worker 的资源切片，并连同精确的扣减一起记录批准；批准保留该请求的交易、准入向量和直接输入证书（第 IV-D 节）。即使每个本金输入都是最终输入，CAL 预留量也等于含找零在内的 a_T ，因为每个已认证输出都可能在支付执行之前流通。成员在该事务提交之后才签名，重试复用同一份批准，释放资源额度时消费锁仍然保留。

收集者向四名成员发送一份规范请求，并在验证了对同一 c_T 的三个签名后返回；不同的有效子集给出同一个事实。收款方对照输出体、索引、所有者和受信任的配置检查 TXCer，原子地记录该币，并可以立即构造后继请求，请求携带其直接输入证书，而不是祖先链。

中继材料在后台保存，有界的早期任务使 **INSTALL** 和公共提交可以各自独立地进行。**INSTALL** 在成员处存储完整的材料，并在本地记录已认证的消费；它既不产生新的批准，也不产生预算扣减。公共提交则单独进行。成员相对于自己首次 **INSTALL** 的时间来安排备份提交，重复 **INSTALL** 不能推迟它。**HTTP** 接受并不移除 **outbox** 条目，该条目保持待处理，直到经认证的公共完成将其移除。备份提交要求保留有完整的提交；第 IV-D 节增加了一条基于原签署者批准记录的路径。

公共的 **DirectSubmission** 携带交易、准入向量、组织的 **SpendQC** 和直接输入证书。委员会重建 Σ_T 并验证同样的授权；省略单独的新输出 **TXCer** 封装只改变信封，而保留该要求。公共验证把每份输入证书绑定到其网络、签发组织、事实和被声明的输出，并拒绝重复。它对每个证书输入恰好接受一份证书，并拒绝任何多余的证书。准入向量含一个 **CAL** 条目，绑定到签发组织。

B. 带直接义务的原子执行

公共执行把整份证书的覆盖与实际的缺失输入义务区分开来。在某个父事实 c 第一次出现缺失使用时，**register** 预留其全部经认证的 **CAL** 金额，并为每个输出创建一个 **Promise**。覆盖记录维持

$$\text{Original}_c = \text{Discharged}_c + \text{Paid}_c + \text{Remaining}_c, \quad (2)$$

其中 Paid_c 是总额累计计数器， $\text{Rec}_c \leq \text{Paid}_c$ 统计由源交易执行偿还的部分。授权额度用量满足 $\text{Reserved}_g + \text{Spent}_g \leq B_g$ ，其中 Spent_g 对 g 之下各证书的 $\text{Paid}_c - \text{Rec}_c$ 求和。登记会提高授权额度的公共用量，而不触动备付金账户；一次有效的赔付（第 IV-E 节）扣减该账户并提高 Spent_g ，一次偿还则向该账户记入并把 Spent_g 降低同样的数额。只有实际被消费的缺失输出才会开立直接义务并提高公共缺口，所以消费一份 40/60 证书中缺失的那个 40-CAL 输出，会预留 100 CAL、开立 40-CAL 的缺口，并使未使用的 **Promise** 仍处于覆盖之下。

图 2 给出公共转换。其前置状态包含同一区块内先前已成功的命令；静态授权可以缓存，而消费、余额、授权额度和义务则对照当前状态求值。所有变更都发生在私有 **overlay** 中，失败会丢弃整个转换。

义务绑定被消费的输出、父证书事实、签发组织、金额、消费交易和历史输入位置，其期限以公共区块时

Require: 已验证的提交 T 、当前状态 S 、公共时间

- 1: 若 T 已处于 **settled** 状态，返回无新增效果
- 2: $O \leftarrow \text{Overlay}(S)$
- 3: 检查意图、授权额度和工作资源；在 O 中为费用托管注资
- 4: **for** 每个 **CAL** 输入 (x, i) **do**
- 5: 要求 (x, i) 尚未被消费
- 6: **if** 存在 (x, i) 的最终创建记录 **then**
- 7: 对照声明检查其输出体和证据
- 8: **else**
- 9: 要求 $i = 0$ 且存在匹配的直接 **TXCer** c
- 10: 若 c 的覆盖尚未登记，则登记其全部覆盖
- 11: 创建唯一的 O_x ；增加缺口和 **Pending**
- 12: **end if**
- 13: 在 O 中记录 (x, i) 的消费
- 14: **end for**
- 15: 要求 **CAL** 输入总值与输出总值相等
- 16: 记录 T 为 **settled**；推进其费用阶段
- 17: 解析每个输出：履行 **Open** 义务；对 **Repaired** 义务向签发组织偿还并标记为 **Recovered**；否则履行已登记的 **Promise**。除非已偿还，否则创建最终的实例 0
- 18: 把全部变更和结果作为一个转换返回

图 2. 公共支付执行。任一检查失败都会拒绝全部暂定变更。输出解析遵循第 IV-C 节。

间为锚。消费方支付记录其 **Pending** 的输入义务和费用进度。用户出资的费用消耗经认证的最终 **FUEL**，把多余的输入价值作为找零退回，并托管所签署的最大值，该最大值涵盖登记、结算、关闭，以及每个已认证输入的一次赔付费用；唯一的阶段标识和义务标识防止重复收费，未用完的托管在输入义务关闭时退回付款方。即使某些输入仍是义务，子输出也是最终输出，因此后续交易按普通的最终输入检查来消费它们。

C. 源交易关闭、偿还与累计对账

源交易通过正常的支付执行到达，其中包括对它自己的输入的验证和消费，其输出在该转换内按各自的义务解析。到达把 **Open** 义务关闭为 **Fulfilled**：它履行 **Promise**，解除覆盖，并降低消费方的 **Pending** 计数。没有任何支付消费过的输出，履行已登记的 **Promise**，并成为最终的实例 0。**Repaired** 义务意味着由一笔备付金扣减为消费者提供了资金，因此到达时偿还该扣减，而不是创建第二个输出：该金额返还给付出它的签发组织账户，义务变为 **Recovered**，偿还计数器和授权额度的 **Spent** 同时变化，而赔付决定仍保持原有记录。已被消费的实例 0 在每个分支中都保持已消费；已偿还的输

出没有最终创建记录。源交易自己的输入，可以在同一个原子转换内对其直接签发组织开立新的义务。

成员依据经认证的连续区块和结果重建进度，并且只有已存在的本地批准才提供可恢复的扣减。对于其 CAL 扣减，成员 m 计算

$$t_{m,T} = \begin{cases} a_T - (\widehat{p}_{m,T} - \widehat{\text{Rec}}_{m,T}), & \text{if settled,} \\ 0, & \text{otherwise,} \end{cases} \quad (3)$$

$$\delta_{m,T} = t_{m,T} - \text{Applied}_{m,T},$$

其中“settled”表示该成员已观察到自己的支付成功执行， $\widehat{p}_{m,T}$ 和 $\widehat{\text{Rec}}_{m,T}$ 是为该批准的输出在本地记录的赔付与偿还。观察到自己的支付执行时，成员检查经认证的结果中的偿还额覆盖 $\widehat{p}_{m,T}$ ，并令 $\widehat{\text{Rec}}_{m,T} = \widehat{p}_{m,T}$ ，因为该次执行偿还了 T 的每个已赔付输出；在跟随赔付之后才批准 T 的成员，则未记录任何赔付。检查边界和单调性之后，它在原 Worker 中把 $\delta_{m,T}$ 从 Reserved 移到 Available，并更新 Applied。处于 Pending 的输入不影响 CAL 释放，因为这些负债由其签发组织承担；工作资源则等待费用关闭。重复的区块和迟到的材料不能重复记入，未执行的源交易只能通过它自身的执行获得释放。钱包把已偿还的输出记为终态观察，没有证书、最终创建记录或余额。

一次经授权的备付金增加，会扣减外部的非保护 CAL 账户，向备付金记入，并原子地提高其授权额度。每个成员加上新旧 $\lfloor 2B_g/3 \rfloor$ 份额之差，同时 Reserved 及其批准保持不变，汇总的支撑检查涵盖共享同一账户的所有授权额度。

D. 基于公共证书的源交易补交

执行一笔支付的区块，携带认证其已认证输入及这些输入的创建支付的法定人数证书。每个成员在其经认证的区块事务中跟随该区块。对于所携带的每份属于自己组织的证书，它加载以该证书的事实为索引的批准，除非它已观察到该源交易的公共执行，或者已持有该事实的 outbox 条目；并检查所存储的交易和准入向量能够重现该事实。随后它把所存储的交易、所存储的直接输入证书和已公开的 QC 组装起来，应用普通的支付验证。验证成功时，它在同一次原子本地更新中写入 outbox 条目，按成员编号错开首次尝试；已有中继在本地提交完成后投递该条目。其他组织的支付触发同样的路径，未通过这些检查的已存储记录属于损坏状态，会使跟随者停止。

该任务补交的是该组织所认证的那笔支付，因此不需要新的所有者签名或成员签名，不产生批准，输入锁保持不变，也不预留额度。已有的 outbox 条目保持其重试时间，源交易的执行会移除该条目。这条路径需要持有批准的原签署者、跟随链并能够联系委员会；期限届满时仍然缺失的源交易则转入赔付。第 V-B 节表明，只要义务已公开，就至少有两名诚实签署者持有该材料。

E. 赔付决定

期限之后，Open 义务通过一条公共的 CompensationDecision $\delta = (net, x, h, j, k)$ 关闭：其中依次是网络、被消费的输出，以及消费方支付的高度、交易索引和输入索引。该定长的规范编码既不含提交者签名，也不含适配数据。权限来自公共义务，目标坐标由已提交的状态确定，所以对于给定的义务，每个提交者形成的字节相同。委员会的 worker 扫描到期索引，并提交到期的决定，无需请求适配份额。

图 3 给出转换。执行器在读取任何历史区块之前，先把请求的网络、位置和输入槽位与义务比对，因此不匹配的请求无需加载历史即被拒绝。随后它读取位于 (h, j) 的规范消费支付，检查其身份、事实、输入和声明的金额，并要求 x 具有 OriginalFunding 及有效的 opening，同时有一个等宽的备付金替换；它不计算替换 opening，也不联系适配器。一个 overlay 承载全部变更：备付金扣减、Repaired 状态、Paid、Spent、缺口、消费方的 Pending 计数、赔付费用（从消费方的托管转入奖励），以及在不再有输入义务时消费方的费用关闭与退款，同时还有不可变的决定记录 $\mathcal{D}_x$ （义务快照和备付金扣减标识）与待历史表示条目。

结果携带单一的逐输出效果，其中指明父事实和消费方事实、金额、输入位置和扣减标识，以及任何费用输出。重复的相同决定返回一个变更集为空的未应用结果，源交易偿还之后亦然；与记录不同的决定则被拒绝；源交易一旦执行，就不会再为其义务提交新的决定。成员连同游标一起，从经认证的结果应用该效果，并定位原始批准以更新 Paid 和 Pending，钱包则对每个费用输出应用一次。赔付不向收款方或已执行的后继交易记入任何本金。

F. 历史表示命令与批处理

一个 RepairBatch 针对一个历史区块，按规范顺序含 1-32 个条目，每个条目指明一个输出、一个交易

Require: 决定 $\delta = (net, x, h, j, k)$ 、状态 S 、执行高度与区块时间

- 1: 要求 h 早于执行高度
- 2: 若 $\mathcal{D}_x$ 已存在, 则当它与 δ 相等时返回无效果; 否则拒绝
- 3: 要求 $\mathcal{O}_x$ 处于 **Open**, 且消费方位置为 (h, j, k) 、网络为 net
- 4: 要求 $\mathcal{O}_x$ 的期限已被区块时间达到
- 5: 加载位于 (h, j) 的规范消费支付; 检查其身份、事实、输入和声明的金额; 要求 x 具有 **OriginalFunding** 及有效的 **opening**, 并有一个等宽的备付金引用
- 6: $O \leftarrow \text{Overlay}(S)$
- 7: 在 O 中: 扣减签发组织备付金; 把 $\mathcal{O}_x$ 设为 **Repaired**; 将其 **Promise** 转为 **Paid**; 更新 **Spent**、缺口以及消费方的 **Pending** 计数和费用阶段
- 8: 在 O 中: 记录 $\mathcal{D}_x$ 和待历史表示条目
- 9: 返回变更和赔付结果

图 3. 赔付决定。该转换不读取也不请求适配材料。任一检查失败都会拒绝全部暂定变更。

索引、一个输入索引和一个新的输入 **opening**; 输出和槽位互不重复。该批处理固定基础修订以及替换前后的规范区块体摘要, 携带分片 **opening**, 并把逻辑修订从 **Base** 推进到 **Base + 1**。单个 **RepairInput** 是其单条目形式, 并携带被替换的交易字节。两种命令都只接受已有已提交赔付决定的输出, 从决定记录中取得金额、签发组织和替换引用, 并且不写入任何记账状态。

构造者独立于物理安装, 读取最新已提交的逻辑表示, 并选出其槽位仍为 **OriginalFunding** 的已决定输出; 赔付决定在提交时已强制检查可负担性, 因此构造者不模拟任何经济步骤。对每个条目, 它由网络和被消费的输出推导出备付金扣减标识, 并把该槽位的 **OriginalFunding** 连同其 **opening** 替换为备付金引用, 把同一交易的各槽位修补到同一份副本中。序列化长度和所有非目标字节 (包括先前已授权的替换) 保持不变。输入适配按槽位进行, 合并后的区块体所触及的每个区块分片只适配一次, 未改变的分片保持其承诺和证明。

份额端点绑定到固定的委员会身份, 每个持有者根据自己已提交的决定推导所请求的替换, 并拒绝没有对应决定的槽位。收集者通过实际的密码学合成来测试有界数量的三成员候选集合, 仅当所有受影响的分片都验证通过时才接受某个候选, 因此格式错误或无效的响应会淘汰候选, 而其他成员的三份有效贡献仍然可用。一次成功的合成会取消尚未完成的请求。**opening** 提供承诺兼容性, 已提交的决定提供授权。

公共执行对照当前状态重复所有检查 (图 4)。基

Require: 批处理 B 、状态 S 、执行高度

- 1: $b \leftarrow \text{BatchID}(B)$; 若存在成功的 Task b , 返回无新增效果
- 2: 加载 **Canonical**; 要求目标高度更早, 且所述基础摘要与前一摘要成立
- 3: 检查条目顺序, 以及输出和槽位互不重复
- 4: 对每个条目, 要求在所述位置存在决定 $\mathcal{D}_x$, 且规范槽位持有 x 的 **OriginalFunding**
- 5: 重建被允许的合并资金替换并验证其 **opening**
- 6: 要求规范候选区块体摘要等于 $B.\text{Next}$
- 7: 验证非目标字节、长度, 以及区块哈希和分片集合头与所存储的原始身份相等
- 8: $O \leftarrow \text{Overlay}(S)$; 移除待历史表示条目并记录逐输出引用
- 9: 向 O 添加一个逻辑修订、Task 和队列条目
- 10: 返回该转换和一个无经济效果的结果

图 4. 原子的历史表示批处理执行。失败会拒绝整个批处理, 只有成功的转换才会提交。批处理标识覆盖完整的规范命令, 包括其证明数据。

础修订变化、某个条目没有决定, 或某个槽位不再持有 **OriginalFunding**, 都会拒绝整个批处理, 而完全相同的成功重放会在基础验证之前被识别。由于每条命令都要求其槽位中为 **OriginalFunding**, 不同的批处理或单条目命令都无法对同一槽位替换两次, 命令标识和修订标识因所选的打包方式而异。经认证的结果指明该命令以及它是否已应用; 跟随者把它映射为空的经济效果, 赔付的经济效果只来自决定结果。偿还属于源交易自己的执行结果。

G. 保持身份的安装与重放

输入承诺保持交易身份, 分片承诺保持完整区块身份 (H_b, P_b), 其中 P_b 含分片数量和 **Merkle** 根。被改变的分片获得指向其原始承诺的有效 **opening**, 原始共识分片使用 **opening 1** 和修订 **0**。候选、锁定和提交检查比较的是完整区块身份, 而不只是稳定的头部哈希。

公共修订和 Task 授权一个确切的新区块体及其分片证据, 安装者在获取物理修订锁之前, 从一个已提交的读视图中复制它们。对于目标修订 r , 较低的物理版本被验证并安装, 相等的版本要求材料完全相同, 较高的版本则覆盖旧任务而不回滚。原始区块被保留, 分片和本地版本元数据以原子方式写入。安装可以跳过中间的物理版本, 并保留每条经济命令。队列进度保持全局且连续: 对于条目 A_1, B_1, A_2 , 在处理 A_1 期间安装 A_2 会使 B_1 保持未完成。逻辑执行读取已提交的 **Canonical** 修订, 因此每种安装调度都留下同样的经济历史。当修

订后的区块体已在某副本上完成物理安装时,称该修订已在该副本上物化。

重放使用原始区块(其命令包含赔付决定),随后是历史表示命令,并重建该公共命令序列的应用哈希。已有的后继交易保持其授权、输出引用和区块身份。赔付决定作为追加式公共记录,即可在经济上关闭义务。历史表示则增加了第二种能力:位于消费方坐标处的已存储区块指明指定输入的备付金扣减,并且仍可对照原始承诺验证,所以读者可以在区块本身中核对该输入的资金来源,而不必通过决定索引。修订后的区块体记录赔付时由哪份备付金为该输入提供资金,之后的偿还则作为源交易结果中的一个独立事实保留。批处理在一个区块内共享历史表示的工作,直接安装则合并历史写入,二者都不会给正常认证增加一轮。补充方法部分把这种历史表示与单独的决定索引进行了比较。

V. 安全性分析

我们在第 III-A 节的模型下,分析第 IV 节所述协议的有限执行。这些论证针对转换函数的守卫条件展开。除故障与状态假设外,它们还使用第 IV-A 节的验证规则:输入证书绑定网络、组织、事实和被声明的输出,并与证书输入一一对应;每个准入向量含一个绑定到签发组织的 CAL 条目。历史表示只能通过已提交的命令生效。该命令的守卫条件要求赔付决定已经存在,由决定推导出槽位和备付金引用,并验证每个非目标字节和完整区块身份;因此适配输出只是一个候选,它是否可得,仅决定历史表示的进展。共识部分的论证把 Tendermint 的锁定接口 [18] 特化到完整区块身份上。

A. 原始执行与已认证消费

polka 指同一轮中针对某个完整区块身份或 nil 的法定人数 *prevote*。

引理 1 (原始执行一致性): 在共识接口的锁定与有效性规则下,每个已提交高度只接纳一个原始区块体,包括其应用上下文。

证明: 在原始 opening 1 下,分片叶子对与上下文绑定的消息代表元求哈希,因此绑定性与 Merkle 证明在原始分片集合头之下固定了该分片的字节。候选区块体或由分片重建,或在本地与分片配对;Locked 与 Valid 值移动时,区块体与分片一并转移;提案、proof-of-lock (POL)、锁定和提交的比较都使用 (H_b, P_b) 。下载目标一旦改变,旧区块体即被清除;轮次与步骤的守卫条件使每一轮只允许一次 precommit。

同一轮中相互冲突的提交,需要某个诚实节点做出两次 precommit。考虑第 r 轮对 v 的一次提交及其两名诚实签署者,再取其后最小的、存在冲突 *polka* 的轮次 s (包括授权解锁的 nil *polka*),并取该 *polka* 的证书最早形成的时刻。法定人数与这两名签署者相交,而相交的那名签署者的投票需要一个更早的解锁理由,这与 s 或形成事件的选取相矛盾。对已锁定值的投票,沿更早的锁定轮次可追溯到提案验证;Finalize 紧随与完整区块身份相匹配的提交。同步与重放保持区块体、高度和时间不变,补充材料展开了这一论证。 ■

引理 2 (经认证的唯一消费): TXCer 固定其输出和签发组织。任何两份有效证书都不会授权对同一 (x, i) 的冲突消费,并且公共执行至多消费该实例一次。

证明: 绑定性固定了已认证的摘要和经所有者授权的交易,固定的路由又把相互竞争的请求送到同一个组织。两个三成员的法定人数至少相交于两名成员,因此其中必有一名诚实成员同属二者。该成员的原子输入锁先于签名,所以对两个冲突请求,它只会批准其一;预算释放时锁仍然保留。最终输入与证书输入的查询同一个花费键。重放一个事实既不增加消费,也不增加费用。已偿还的输出没有最终创建记录,其实例 0 仍保持已消费,所以此后对 (x, i) 的声明会失败。 ■

B. 源交易补交与赔付决定

对于输出 x , $\mathcal{O}_x$ 是它的直接义务,状态为 Open、Fulfilled、Repaired 或 Recovered, $\mathcal{D}_x$ 是它不可变的赔付决定。Repaired 表示已在经济上赔付,无论已存储字节处于何种状态。决定在从 Open 到 Repaired 的转换内写入,且从不删除,所以 $\mathcal{D}_x$ 恰好在 $\mathcal{O}_x$ 为 Repaired 或 Recovered 时存在。集合 $\mathcal{R}$ 保存历史表示状态(已提交的修订、头命令、任务、批处理引用、队列、待历史表示索引),投影 π_{econ} 保留公共资产、消费、覆盖、义务、授权额度用量和费用状态,而省略 $\mathcal{D}$ 和 $\mathcal{R}$ 。

引理 3 (补交保持授权): 设某诚实成员为事实 c 创建一个补交任务。该任务提交该成员在为 c 签名之前所存储的支付、准入向量和直接输入证书,并附上 c 的公共 QC; 公共验证与执行把它当作该支付的任何一次提交来处理。它不需要新的所有者签名或成员签名,不产生批准,输入锁保持不变,也不预留签名额度。每个成员对每个事实至多持有一个任务,支付一经执行即移除该任务。一旦某个 c 的输出上的直接义务成为公开的, c 的至少两名诚实签署者就保留着所存储的材料。

证明: 成员以公共 QC 中的事实为索引查找自己的批准, 所存储的支付和准入向量必须能够重现该事实。它把这些材料和所存储的直接输入证书与已公开的 QC 组合起来, 在创建 **outbox** 条目之前应用普通的支付验证。事实绑定和法定人数验证排除了任何其他的支付核心内容; 法定人数签名则固定了公开方所选择携带的那份证书的内容。该转换只写 **outbox**, 所以批准、输入锁和授权额度的预留保持原样, 已有的 **outbox** 条目与已观察到执行的标记会抑制重复。未通过这些检查的已存储记录不在故障模型之内。

义务只产生于某个已执行的、消费了已认证的缺失输入的支付, 而公共验证要求提供该输入的证书, 因此该证书是公开的。其三名不同的签署者中至少有两名诚实成员, 每一名都在返回签名之前存储了经批准的材料。批准中不含法定人数证书, 该证书随区块到达, 所以扣留证书的客户端并不会损坏用于补交的材料。执行还需要一个持续运行的诚实跟随者、公平的投递, 以及满足普通执行规则的源支付。 ■

引理 4 (赔付至多生效一次): 每个直接义务至多发生一次备付金扣减及其相关的经济效果。对 x 的一项决定, 若已有记录 $\mathcal{D}_x$, 则当它与该记录相等时无效果 (包括偿还之后), 否则被拒绝。没有记录的决定, 在 $\mathcal{O}_x$ 不再处于 **Open** 时被拒绝。

证明: 状态只沿 **Open**→**Fulfilled**、**Open**→**Repaired** (仅由一项决定完成) 和 **Repaired**→**Recovered** (在源交易的成功执行之内) 变化。决定首先检查 $\mathcal{D}_x$: 完全匹配则返回变更集为空的未应用结果, 不匹配则被拒绝。没有记录时, 该转换要求义务处于 **Open**、期限已被区块时间达到, 以及历史位置、金额和资金承诺相匹配, 这些都从已提交状态重新推导。随后它在一个原子步骤中扣减签发组织的备付金、关闭缺口、更新费用与覆盖, 并记录 $\mathcal{D}_x$ 。状态机是无环的, 所以 **Open** 只会被离开一次, 至多一项决定生效。

若源交易先执行, 义务离开 **Open**, 针对它的决定不再提交。若赔付在先, 源交易的执行通过唯一的 **Repaired**→**Recovered** 转换把资金返还给签发组织, 保留 $\mathcal{D}_x$, 且不创建用户输出。对公共命令顺序做归纳, 即得赔付与偿还至多发生一次。 ■

C. 覆盖公开与隐藏的承诺

每个已认证的事实都带有一个绑定到签发组织的 **CAL** 条目, 因为至少两名诚实签署者在签名之前就通过准入规则计算了该向量, 而事实绑定摘要。对于一个完整的 **CAL** 授权额度 g , 累计授权 B_g 给每个成员 $s(B_g) = \lfloor 2B_g/3 \rfloor$ 的签名额度。令 C_g 包含到当前前缀为止、其三个签名已经存在的每个不同事实, 包括未汇集的与隐藏的证书, 以及已 **settled** 的事实。唯一的 **CAL** 分配把每个事实绑定到一个配置、**ResourceKey** 和 **Grant ID**。事实 c 有输出上限 a_c , 即其 **CAL** 输出之和。对 $c \in C_g$, 令 p_c 为这些输出累计支付的赔付额, Rec_c 为由源交易执行偿还的部分, z_c 表示 c 的支付已成功执行。净赔付额为 $\bar{p}_c = p_c - \text{Rec}_c$ 。定义

$$w_c = \begin{cases} a_c, & z_c = 0, \\ \bar{p}_c, & z_c = 1, \end{cases} \quad Q_g = \sum_{c \in C_g} w_c. \quad (4)$$

引理 5 (源交易偿还): 被赔付的输出只会在其源交易的成功执行之内离开 **Repaired** 状态, 且与置 $z_c = 1$ 发生在同一转换中; 偿还记入当初被扣减的账户。因此 $0 \leq \text{Rec}_c \leq p_c \leq a_c$, 当 $z_c = 0$ 时 $\text{Rec}_c = 0$, 一旦 $z_c = 1$ 则 $\text{Rec}_c = p_c$ 。特别地, 源交易待决时 $\bar{p}_c = p_c$, 此后 $\bar{p}_c = 0$, 所以源交易执行之后不留任何净赔付。

证明: 每个已登记的 **Promise** 至多离开 **Open** 一次, 并且由引理 4, 赔付对一个输出至多生效一次, 且只在其义务处于 **Open** 时生效, 所以 $p_c \leq a_c$ 。离开 **Repaired** 的唯一转换, 是某笔支付的输出解析步骤, 该支付的事实和签发组织与该义务的事实和签发组织相同。它运行在把该支付记录为 **settled** 的 **overlay** 中, 所以 z_c 与偿还一同变化。该步骤解析源交易中每个带有义务的输出, 把 **Open** 变为 **Fulfilled**, 把 **Repaired** 变为 **Recovered**, 此后 c 的每个 **Promise** 都已关闭, 因此不再产生新的赔付, 重复提交为空操作。赔付扣减与偿还记入使用义务中存储的签发组织账户, 也就是 c 的唯一 **CAL** 分配所对应的账户, 而偿还不动 $\mathcal{D}_x$ 。 ■

对任意本地批准, 令 $r_{m,c} = a_c - \text{Applied}_{m,c}$, 其中 **Applied** 是其累计的 **CAL** 恢复量。

引理 6 (剩余额度): 对每个事实 $c \in C_g$ 和每个批准过 c 的诚实成员 m , 在每个前缀上都有 $r_{m,c} \geq w_c$ 。

证明: 在 m 观察到 c 的支付成功执行之前, 它对 c 的 **CAL** 恢复为零, 所以无论赔付状态如何, $r_{m,c} = a_c \geq w_c$ 。观察到该次执行之后, m 检查所报告的偿还额覆盖其记录的赔付额, 并令其记录的偿还额等于该赔

付额。恢复目标是 a_c 减去为零的已记录净赔付，所以 $r_{m,c} = 0$ ，而由引理 5， $w_c = \bar{p}_c = 0$ 。在跟随赔付之后才批准的成员没有记录赔付，只恢复它自己的扣减。■

定理 1 (潜在负债均有资金支撑): 对每个 CAL 授权额度, $Q_g \leq B_g$ 。对每个备付金账户 x ，其未关闭缺口总额 D_x 不超过其余额 A_x 。

证明: 令 $\mathcal{A}_{m,g}$ 包含 g 之下所有成功的批准 (包括没有 QC 的批准)，并令 $L_{m,g} = \sum_{c \in \mathcal{A}_{m,g}} r_{m,c}$ ，跨 Worker 求和。批准和恢复在 Available 与 Reserved 之间移动额度，充值则加上成员份额之差，所以对本地观察到的授权 $B_{m,g}$ 有 $L_{m,g} \leq s(B_{m,g}) \leq s(B_g)$ ；授权额度只会增加。任取该组织的三名诚实成员 H_g 。一份 QC 有三名不同的签署者，且至多一名成员位于 H_g 之外，所以 H_g 中签署过 c 的成员集合 Sig_c 至少有两个元素。诚实成员只在批准之后才签名，因此引理 6 对 $m \in \text{Sig}_c$ 给出 $r_{m,c} \geq w_c$ ，于是

$$\begin{aligned} 2Q_g &\leq \sum_{m \in H_g} \sum_{\substack{c \in C_g \\ m \in \text{Sig}_c}} r_{m,c} \\ &\leq \sum_{m \in H_g} L_{m,g} \leq 3[2B_g/3] \leq 2B_g. \end{aligned} \quad (5)$$

令 R_c 与 d_c 分别为已登记的剩余覆盖和已解除覆盖，登记之前 $p_c = d_c = R_c = 0$ 。已登记的记录满足 $a_c = p_c + d_c + R_c$ ，且 $z_c = 1$ 蕴含 $R_c = 0$ 。因此 $R_c \leq w_c - \bar{p}_c$ ：当 $z_c = 0$ 时 $\text{Rec}_c = 0$ ，所以 $R_c \leq a_c - p_c = w_c - \bar{p}_c$ ；一旦 $z_c = 1$ ，两边都为零。定义 $S_g = \sum \bar{p}_c$ 、 $V_g = \sum R_c$ 和 $E_g = Q_g - S_g$ ，求和范围均为 C_g 。登记、赔付、偿还和解除维持公共的 Usage.Spent = S_g 与 Usage.Reserved = V_g 。

令 $\mathcal{G}_x$ 包含由 x 支撑的所有授权额度；它们经认证的 CAL 账户就是被扣减的签发组织账户。创世状态保证 $\sum_{g \in \mathcal{G}_x} (B_g - S_g) \leq A_x$ 。外部的、非保护的充值使两边同等增加。赔付使 S_g 增加、 A_x 减少同样的数额，偿还使 S_g 减少、 A_x 增加同样的数额 (引理 5)，没有其他协议操作会提取 CAL 备付金。每个 Open 义务对应一个互不相同的剩余 Promise，所以

$$\begin{aligned} D_x &\leq \sum_{g \in \mathcal{G}_x} V_g \leq \sum_{g \in \mathcal{G}_x} E_g \\ &\leq \sum_{g \in \mathcal{G}_x} (B_g - S_g) \leq A_x. \end{aligned} \quad (6)$$

总额 Paid 与其偿还额各自只计算一次，且 E_g 是风险上界，而不是成员可用的额度。■

CAL 准入推论。 在执行前置状态 (包括同一区块内先前成功的命令) 下，令 N_g 为 g 之下互不相同的、有效的、尚未登记的父事实，这些父事实是匹配的、未被消费的缺失输入所要求的。它们满足 $w_c = a_c$ 和 $p_c = R_c = 0$ 。每个已登记的事实都满足 $R_c + \bar{p}_c \leq w_c$ ，且这些事实互不相交，所以

$$V_g + S_g + \sum_{c \in N_g} a_c \leq Q_g \leq B_g. \quad (7)$$

对同一证书的重复使用只计一次，顺序登记能看到先前的预留，所以每个部分和都满足该上界。因此，首次登记不会因 CAL 授权额度不足而失败，即使它在同一转换中先于一次偿还，因为偿还只会降低 S_g 。同样，一笔 Open 金额 $a \leq D_x \leq A_x$ 有足够的备付金本金。其他资源和费用检查仍然另行进行。

D. 资产守恒与延迟中性

定理 2 (CAL 守恒): 令 U 为最终的、未花费的 CAL 实例的总价值， A 为 CAL 账户余额总和， D 为 Open 缺口的总价值。每个执行前缀都满足

$$U + A - D = C_0, \quad (8)$$

其中 C_0 包括创世输出和所有账户 (含充值来源)。

证明: 一笔支付消费最终价值 F 与缺失价值 M ，并产生 $F+M$ 。令 J 和 R 分别为其到达输出中义务原为 Open 和已被赔付者的总价值。前一类输出已被消费，所以它们关闭缺口而不增加未花费价值；后一类输出没有创建记录。因此 $\Delta U = M - J - R$ ， $\Delta D = M - J$ ，偿还给出 $\Delta A = R$ ，故 $\Delta(U + A - D) = 0$ 。金额为 a 的赔付决定给出 $\Delta A = \Delta D = -a$ 和 $\Delta U = 0$ 。补交不改变任何公共状态，并且由引理 8，历史表示命令和物理安装使 U 、 A 、 D 保持不变。充值是在被纳入的账户之间转移，其他转换保持该投影。■

FUEL 遵循单独的记账: 每个托管最大值等于 Held 加上累计的 Rewards、Burned 和 Refunded；全局守恒和统计未花费 FUEL、普通账户、当前 Held、奖励余额和累计销毁量。唯一的阶段标识和义务标识使价值只转移一次，用户出资与组织出资的费用皆然。

守恒确定总量，下一个结果则确定最终持有者。

定理 3 (延迟中性): 设 $\mathcal{T}$ 是一个有限的支付集合，满足：(i) 没有两笔支付消费同一实例或同一费用输入；(ii) “消费某支付所创建的输出”这一关系是无环的；(iii) $\mathcal{T}$ 中每笔支付的每个证书输入，都认证 $\mathcal{T}$ 中某笔支付

的一个输出。设 $\mathcal{U}$ 是一个有限的经授权的备付金增加集合。从干净的创世状态出发，其中没有覆盖、Promise 或义务记录，且 CAL 的 Reserved 与 Spent 为零。令 ρ_0 按与 (ii) 一致的顺序执行 $\mathcal{T}$ ，并应用 $\mathcal{U}$ ；令 ρ 为由 $\mathcal{T}$ 的支付、 $\mathcal{U}$ 的增加以及到期的 Open 义务的赔付决定组成的任意执行，顺序任意，并可与可接纳的历史表示命令任意交错。假设每笔支付和每次增加在两个执行中都恰好成功一次。则在 ρ 和 ρ_0 结束时：

- 1) 最终的、未花费的 CAL 实例相同，所有者和金额也相同；
- 2) 每个备付金账户持有相同的 CAL 余额；
- 3) 每个 CAL 授权额度具有相同的授权，且 $\text{Reserved} = \text{Spent} = 0$ ，每个义务为 Fulfilled 或 Recovered，并且每份证书满足 $\text{Paid}_c = \text{Rec}_c$ 。

费用和 FUEL 不作比较。

证明：持有者。由 (i)，两个执行消费相同的实例。在 ρ 中，在其源交易执行之前被消费的输出会获得一项义务；当源交易执行时，该输出要么是 Fulfilled，其创建记录已被消费，要么是 Recovered，没有创建记录；两种情形下它都是已被消费的。没有被 $\mathcal{T}$ 中任何支付消费的输出，在两个执行中都被创建为最终的、未花费的。因此未花费的实例就是创世输出，以及 $\mathcal{T}$ 的输出中 $\mathcal{T}$ 未予消费的那些。

备付金。对备付金账户 x ，令 $\hat{A}_x$ 为其余额加上由 x 签发的 Repaired 义务的金额。赔付使 A_x 减少，并使 Repaired 总额增加同样的数额；偿还在同一账户上反向进行（引理 5）；创建或履行义务则不改变这两项。如同定理 1，只有充值、赔付和偿还改变备付金余额，所以在每个前缀上 $\hat{A}_x = A_x(0) + \tau_x$ ，其中 τ_x 是迄今记入 x 的增加总额。义务只对在其源交易执行之前被消费的输出存在，并且由 (iii)，该源交易在 ρ 中执行，并使该义务变为 Fulfilled 或 Recovered。结束时没有义务处于 Open 或 Repaired，所以 $A_x = A_x(0) + \tau_x$ ，这也是 ρ_0 中的值。

用量。每份已登记的证书都是因某个缺失输入的使用而登记的，所以由 (iii)，其源交易在 ρ 中执行。该次执行关闭它的所有 Promise，所以 Remaining = 0，并且引理 5 给出 $\text{Paid}_c = \text{Rec}_c$ 。于是 $\text{Reserved} = \sum \text{Remaining} = 0$ ， $\text{Spent} = \sum (\text{Paid}_c - \text{Rec}_c) = 0$ ，授权只通过 $\mathcal{U}$ 改变。

历史表示。由引理 8，插入其间的历史表示命令不改变任何被比较的量，也不改变同一高度和时间的后续支付或决定的结果。 ■

中间状态是不同的：被赔付的源交易的签发组织，在源交易执行之前持有较少的备付金，这由定理 1 涵盖；而被比较的投影省略了只存在于 ρ 中的决定记录和历史表示状态。始终不执行的源交易，会使其被赔付的金额成为其签发组织的净损失，激励问题不在该定理的范围之内。

E. 授权的历史表示与独立性

引理 7 (由赔付决定把关的历史表示权限)：每个有效的历史表示条目都指向一个已存在的赔付决定（包括其义务其后已被偿还的决定），以及一个仍持有 OriginalFunding 的规范槽位。成功的历史表示不会改变其被授权的资金槽位之外的任何交易字节，即使适配输出是對抗性的。

证明：设 N 为 RSA 模数， e 为其公开指数， r 为 $\mathbb{Z}_N^*$ 中的 opening。带域分隔的代表元 $\text{FDH}(ctx, m)$ 绑定消息及其修复上下文。关系

$$C = \text{FDH}(ctx, m)r^e = \text{FDH}(ctx, m')r'^e \pmod{N} \quad (9)$$

给出兼容性，且不带来新的批准；已公布的 opening 比值可以在同一消息关系下复用。公共守卫条件读取决定记录（其中含义务快照和备付金扣减标识），并由它推导槽位和替换引用。它们检查当前的基础摘要和区块体摘要，并验证所有非目标字节及完整的区块承诺。读取快照使已决定的槽位在偿还之后仍可被历史表示。无效的候选不留下任何变更，完全相同的成功重放为空操作，重新打包的命令会遇到已不再持有 OriginalFunding 的槽位或已存在的任务。决定已经提交了经济效果，所以命令不提供金额、签发组织或账户变更，适配器也不写入账本状态；论证只依赖经验证的输出。 ■

引理 8 (历史表示在经济上是惰性的)：对每个成功的单条或批处理历史表示命令 r 和状态 S ，有 $\pi_{\text{econ}}(r(S)) = \pi_{\text{econ}}(S)$ 。在相同高度和时间执行的支付或决定，在 $r(S)$ 中与在 S 中给出相同的结果，且两个后继状态的投影一致。

证明：历史表示命令检查决定、规范目标、基础修订、前一摘要和被允许的替换。其写集包括修订、头命令、任务与批处理引用记录、队列，以及待处理索引的移除；它跳过记账转换，不写入任何账户、义务、覆盖、用量、花费或费用记录。跟随者把其结果映射为空的经效果。因此对单步而言投影不变，并由归纳，对任意此类步骤的序列也不变。

针对输出 y 的决定读取规范区块体时, 只通过交易身份、授权事实、输入、声明和金额、编码长度, 以及 y 的槽位。若 $\mathcal{D}_y$ 已存在, 该决定是不读取任何历史的空操作。否则 y 的槽位仍持有 **OriginalFunding**, 因为替换需要 $\mathcal{D}_y$, 而替换其他定宽槽位不改变其余字段。支付执行从不读取规范区块体。 ■

引理 9 (身份保持与安装无关性): 有效的历史表示保持所有者授权、交易与输出标识、声明、后继交易引用, 以及完整的原始 **BlockID** (H_b, P_b)。对于同一公共历史, 合法的物理安装调度具有相同的经济投影。此外, 若变色龙关系对每个固定的消息与承诺都有唯一的有效 **opening**, 则对于固定的原始区块体和固定的已决定槽位集合, 最终区块体和分片 **opening** 不依赖于历史表示命令的分组或顺序。修订号、命令历史和应用哈希可以不同。

证明: 执行器构造规范区块体时, 只改变已声明的资金槽位及其 **opening**。交易身份、声明和输入承诺由式 (1) 的身份编码保持固定, 替换保持编码宽度不变。执行器验证新的 **opening**, 重新计算修订后区块体的区块哈希和分片集合头, 并且只接受与原始 **BlockID** 相等的结果。Base 与 Previous 把相互竞争的规范修订串行化, 已提交的 Task 绑定被允许的字节。后继交易引用的是交易与输出标识, 所以既不需要重新签名, 也不需要重新执行。

Canonical 读取已提交的逻辑修订 ρ_ℓ , 一个确切的 Task 授权修订 $r \leq \rho_\ell$ 处的区块体和分片。若 $\rho_p < r$ 则安装推进, 若 $\rho_p = r$ 则检查完全相等, 并把 $\rho_p > r$ 视为已被覆盖而不回滚。它不写入经济状态, 只读取已提交的修订和任务记录, 队列进度只越过已完成的条目。对同一公共前缀上的合法调度 μ_1, μ_2 , 这给出

$$\pi_{\text{econ}}(S_{\mu_1}) = \pi_{\text{econ}}(S_{\mu_2}). \quad (10)$$

收敛性陈述来自每个槽位和分片 **opening** 的唯一性, 以及分片上下文与修订计数器无关。补充材料给出这一论证, 以及跟随者更新与历史之间的比较。 ■

F. 受保护的续花与进展

令 **FreshRecv**(w, o, c) 表示由某个此前未授权消费该实例的诚实所有者所做的经认证接收。新鲜性是一个历史谓词, 并不能仅凭接收成功推出。

定理 4 (受保护的续花): 在 **FreshRecv** 之后, 已认证的输出若无所有者授权, 就不能被重新解释或消费。

一个已认证并经公共执行的后继交易, 要么消费一个最终源, 要么原子地建立一项有资金支撑的直接义务。源交易的关闭、赔付与历史表示保持其已确立的本金效果, 无需后继交易重新签名或重新执行。

证明: 引理 2 固定所有权并排除冲突消费, 原子执行把每次缺失消费与其义务和输出耦合在一起。定理 1 提供资金支撑, 定理 2 对关闭和迟到的源交易记账, 引理 1 确定已执行的历史, 引理 3 约束补交, 引理 4 约束赔付, 引理 7-9 约束后续的变更。对本地与公共的联合转换做归纳, 即把这些性质组合起来。INSTALL 保留本地的消费效果, 不改变公共资产。赔付决定可以改变 Pending、费用和备付金余额, 但不重复后继交易的本金; 历史表示和物理安装则不改变其中任何一项。 ■

续花要求诚实法定人数能及时响应、证据可得、费用和 Worker 或工作额度充足, 以及公平的投递与纳入。部分批准和未使用的 Promise 可能继续占用额度, 超时从不授权解锁。这些是没有统一完成期限的进展条件。有三个条件是相互独立的: 源交易的补交、义务的经济关闭, 以及其历史表示的完成。

引理 10 (经济关闭无需等待适配): 一旦某项未关闭的直接义务的公共期限已到、且其赔付决定被纳入, 只要其位置处的原始历史、签发组织的备付金和消费方的费用托管可得, 就可由该公共赔付决定关闭。这一转换不需要任何适配贡献。历史表示的完成则另外需要有效的适配材料、当前的规范修订, 以及公共提交和本地物理安装的进展。

证明: 赔付决定的路径验证义务和历史槽位, 并在不使用适配端点的情况下完成记账。由定理 1, 备付金足够; 准入规则在消费方的托管内为每个已认证输入预留一次赔付费用。因此适配不可用既不阻塞决定, 也不阻塞其后的偿还。门限服务需要来自四名固定持有者的三份有效贡献; 三名响应的诚实持有者就足够, 拜占庭持有者也可能提交有效的贡献。共享的执行资源和公共排序仍然决定延迟, 所以这一陈述只分离协议依赖关系, 不给出完成时间上界。 ■

可用性与担保人风险。 安全性、进展和经济后果是三个相互分离的问题。

安全性。 定理 1 约束所有已认证支付的未清偿 CAL 负债, 包括未公开提交的支付。

进展。 当某个诚实中继续持有源交易的提交材料 (交易、准入向量、组织授权、直接输入证书), 并且费用、额度、法定人数和调度条件成立时, 源交易就会推进,

投递时间不设上界。由引理 3，一旦义务公开，至少两名诚实签署者持有这些材料。在拥塞或短暂中断下，期限仍可能到达。若赔付决定执行时源交易仍然缺席，该决定无需适配即可关闭义务（引理 10）；签发组织垫付缺失的本金，源交易执行时再偿还它。适配贡献可得时，历史表示完成。

经济后果。当一个支付集合的所有源交易都执行时，定理 3 给出不变的最终持有者和备付金。批准端点认证所有者、预算和输入，而不关心调用者的角色，所以任何客户端都可以充当收集者。由于原签署者会补交，一旦后继交易已执行，扣留证书的收集者本身无法使源交易保持未执行。只有当补交的支付由于跟随者不可用、投递或纳入失败，或因其自身的执行规则而未能执行时，源交易才保持未执行，其被赔付的金额才继续是签发组织的损失。备付金的规模反映这类源交易的预期比率，费用不为此提供保险。

VI. 实现与评估

我们评估依赖支付的延迟、备付金的关闭与偿还、证据驱动的源交易解决，以及历史表示的开销。随后，组织路径实验和负载实验刻画快速路径的运行条件。

实现与版本。Go 实现使用原子状态 overlay、独立的区块跟随者，以及经过修改的 CometBFT v0.38.26，后者被用来强制执行完整区块身份和经授权的历史安装。门限 RSA 适配使用 CIRCL v1.6.5，由可信的离线分发者初始化。实验在一台 Mac Studio (M4 Max, 16 个 CPU 核心, 64 GiB 内存, macOS 26.6.2, Go 1.27.1) 上运行。四个委员会进程，加上每个组织的四个成员和一个网关，使这台主机上共有九个或十四个服务进程，且每次运行都从全新的创世状态开始。

结果来自三类版本，其标识见补充材料。基线版本（提交 d99ec92）实现分层续花以及第 IV-B 节的记账；其赔付与历史表示一并提交。*decision-first* 版本实现第 IV-D 节和第 IV-E 节的补交与独立的赔付决定。归档版本提供交付门控、组织路径、运行范围、资本和批处理集成各系列，并在使用处加以标注。在一个版本上测得的数值，不用于另一个版本。

测量约定。认证接收延迟从钱包第一次发出 HTTP 请求开始，到收款方验证并完成原子的本地记录之后结束。公共完成时间和成员完成时间是客户端对经认证执行的观察，包含轮询，不是内部提交时间戳；完成吞吐量包含最终的后台排空。延迟与容量试验主要使

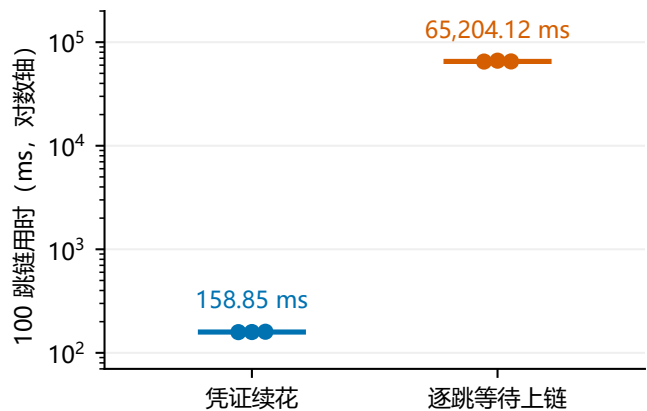


图 5. 基线版本上 100 跳链到最后一次认证接收的耗时。圆点为每种模式的三次独立运行；横线标出它们的中位数。对数时间轴在两种模式中包含相同的终点。只有等待式基线要求在相邻支付之间进行公共确认。

用内存中的成员与委员会后端，以及采用 NoSync 的 bbolt 钱包，同时启用签名和原子状态更新；恢复试验和 decision-first 试验使用落盘的委员会。续花试验与恢复试验使用最终的用户 FUEL，而存储、配对后台和短时吞吐的对照使用经授权的组织代付 FUEL。

A. 无需逐跳结算的连续花费

我们把认证续花与在各跳之间等待公共执行进行比较。在基线版本上，100 跳链到达最后一次认证接收的中位耗时为 158.846 ms，而每一跳都等待公共确认时为 65.204 s。三个钱包轮流转账 100 CAL，每笔支付都由其前一笔的输出在线构造，并用自己的最终 FUEL 支付费用。每种模式重复三次，在九服务的内存配置上共得到六次运行、600 笔支付。两种模式都以最后一次认证接收为终点，等待模式则在各跳之间增加一次公共确认。在委员会 commit、flush 和 gossip 设置分别为 250、10 和 10 ms 时，公共执行及其观察每跳耗时约 0.65 s。认证续花消除了收款方路径上的这段等待。

图 5 给出六次整链耗时。快速运行耗时 158.492–159.905 ms。快速模式下全部 297 个后继请求，都消费其前一笔已认证的输出，并在该输出的认证接收之后、其公共完成被观察到之前发出。一个单输入、单输出的 TXCer 为 779 字节，请求在第一跳为 1,979 字节，之后为 2,766 字节。对于这种交易形态，直接证据的大小在全部 100 跳中保持不变，因为每个请求携带的是其直接输入的证书，而不是祖先链。

为检验这一优势是否依赖于内存中的成员存储，我们在九服务的落盘配置上，使用 NoSync 钱包和用户自

付 FUEL，做了三组配对的 100 跳试验。这些试验仅在成员存储上不同（NoSync 或同步；顺序为 N/S、S/N、N/S），每组之后各跟一条同步的等待链。中位耗时分别是：NoSync 续花 681.127 ms，同步续花 4.471 s，同步等待 65.013 s。因此在成员提交为同步时，认证续花仍比等待快 14.5 倍。全部 900 笔支付完成，且各委员会状态一致。

在归档的交付门控版本上，认证之后有一个投递门控，它等待三名成员 INSTALL 完整材料，或等待经认证的公共执行成功。该门控不收集第二个签名法定人数；改为立即交付会缩短续花。在每秒 200 笔支付、持续 30 s 的条件下，每种模式三次配对运行各完成 6,000 笔支付，运行级接收 P50 的中位数从有门控时的 2.167 ms 降到 1.300 ms（下降 40.0%）。另外的 100 跳试验从 287.983 降到 200.356 ms（下降 30.43%）。授权不变，补充材料列出了配对运行。

B. 有资金支撑的关闭与偿还

本实验检验赔付是否保持后继支付不变，以及每次赔付是否在其源交易随后执行时恰好偿还一次。

赔付与偿还矩阵（基线版本）。九次运行使用随机种子 23、37 和 59，被扣留父交易的比例分别为 0%、1% 和 5%。每次运行按每秒 20 个两跳单元的速率提供负载，持续 60 s（1,200 个单元，2,400 笔支付），对应一份固定的 60,000-CAL 备付金，使用用户自付 FUEL 且不充值。九服务部署使用落盘的委员会应用状态和 BlockStore，成员和网关在内存中，钱包为 NoSync。驱动程序只有在每个被扣留父交易的赔付已在全部四个副本上提交并完成安装之后，才释放该父交易，所以每个被扣留的源交易都在 30-s 公共时间期限之后被赔付，并在其后执行。

全部 21,600 笔支付都完成关闭。216 次赔付（1% 的每次运行 12 次，5% 的每次运行 60 次）各自之后都有偿还。每次运行结束时，没有义务处于 Open 或 Repaired，缺口为零，净 Spent 和 Reserved 为零，备付金回到 60,000 CAL（表 II）。所采样到的最低备付金在 1% 时为 59,800–59,900 CAL，在 5% 时为 59,600 CAL。它反映的是驱动程序的释放节奏，不是备付金规模的结果，补充材料绘出了三次 5% 运行的轨迹。正常子支付的运行级接收 P50 为 1.185–1.251 ms。从期限起算，委员会提交（在该版本中它承载历史表示）的观察时间中位数为 1.003–1.254 s，全部四个副本上完成修订区

表 II

赔付与偿还矩阵的收尾审计。每一列都被三个随机种子复现；每次运行有 2,400 笔支付。备付金数值以 CAL 计，费用数值以千 FUEL 计，费用上限为各笔支付最大值之和。

被扣留父交易比例	0%	1%	5%
赔付 / 偿还次数	0	12	60
赔付 / 偿还 CAL	0	1,200	6,000
最终备付金	60,000	60,000	60,000
未关闭 / 净 Spent	0	0	0
费用上限总额	2,400	2,400	2,400
Rewards	201.600	201.660	201.900
Burned	24.000	24.000	24.000
Refunded	2,174.400	2,174.340	2,174.100
Remaining Held	0	0	0

块体物理安装的时间中位数为 1.257–1.761 s（P95 最高 2.015 s）；两者都包含 250-ms 的观察轮询。每笔正常支付花费 94 FUEL，每次赔付增加 5，奖励、销毁与退款之和等于费用上限，Held 为零。

账本层面的回归测试直接覆盖到达顺序：四跳链的全部 24 种到达顺序，在有到期赔付与无到期赔付两种情形下，各取三种签发组织序列，共得到 144 条轨迹；另有 48 条拆分-合并轨迹，在两种赔付模式下，改变一个 40/60 拆分、两条分支及其合并的全部到达顺序。192 条轨迹中的每一步都保持独立计算的资产投影守恒。每条轨迹结束时，各签发组织的备付金恢复，Spent 与 Reserved 为零，Paid = Rec，已消费的输入仍保持已消费。

两个十四服务的对照实验区分永久损失与延迟偿还。父交易始终不到达时，孙支付保留其 100 CAL，而负责的签发组织的备付金仍然低 100 CAL。当跨组织链以孙支付最先到达的顺序到达时，中间源交易关闭其自己的直接义务，根签发组织被赔付的 100 CAL 在根交易其后执行时得到偿还。两种情况下四个委员会状态一致，缺席的父交易自身的费用和工作预留可能保持未清偿。

三组配对的十四服务运行使用基线版本的初始构建（在批处理结果标签分离之前），每次 4,000 笔支付、速率 100 笔/秒，其正常运行与扣留源交易运行最终的所有者持有和备付金余额完全一致，与定理 3 一致。并发赔付使接收 P95 升高为 1.98–2.21 倍，这些配对不能区分造成这一升高的具体环节。

一个使用 NoSync 钱包的存储对照把成员提交单独

隔离出来：NoSync 成员时接收 P50 为 1.0 ms，同步提交时为 43–52 ms。

早期版本的资本系列（每个条件一次运行、没有赔付）表明，10,800–19,700 CAL 的外部新增，足以维持从 3,600-CAL 起步、各含 6,000 个两跳单元的 300-s 运行，而固定 3,600-CAL 的对照只关闭了 6,000 个单元中的 46 个。

C. 补交、决定与历史表示

我们检验原签署者能否送达被扣留的源交易，适配不可用时赔付与偿还能否继续进行，以及这些机制在普通支付路径上的代价。

设置。 decision-first 版本运行于十四服务配置，使用全新的创世状态，每轮 450 个初始输出，落盘的委员会，成员为默认的 NoSync 模式，flush/gossip 设置为 10 ms。用户支付 FUEL，所有签名、金额和消费检查均保持开启。九次功能运行使用初始实现快照，下文的普通路径运行使用最终快照（见补充材料）。一轮功能运行先认证一笔父支付，在完整 QC 形成之后阻止其原发布者提交或 INSTALL 它，再让一笔子支付花费该父支付的 TXCer。原签署者持有其批准，但不持有完整 QC。共运行三种处理，每种三轮：补交（按设计运行的版本）、对照（关闭自动补交）和暂停（关闭补交，并由一个标记使全部四名委员会成员的适配端点暂停，而共识、决定和普通支付继续）。计时从子支付的钱包 HTTP 发送开始。快速接收在钱包验证并本地记录时结束，公共事件是带轮询的客户端观察。这些两跳演示使用默认钱包模式，因此其接收时间不能与下文 NoSync 压力运行的接收时间相比较。

基于公共证书的补交。在全部三轮中，子支付的快速接收耗时为 13.91–13.94 ms。子支付所在的区块在 1.414、1.990 和 1.440 s 被观察到，被扣留的源交易随后在 2.014、2.565 和 2.039 s 完成公共执行，比子支付晚 575–600 ms，且驱动程序没有提交原支付（图 6a）。每次审计都显示有一个义务被履行（Fulfilled）、没有赔付，且签发组织的备付金未被动用。收款方在约 14 ms 之后即可继续，而约 2 s 是后台源交易被观察到的完成时间。这些时间间隔是轮询粒度下的客户端观察，不是协议延迟的上界，两次公共事件之间的 575–600 ms 也未进一步分解。

无需适配的赔付。关闭补交之后，源交易始终缺席，义务在每一轮中都越过其 30-s 期限。其后的决定在义务仍

处于 Open 时执行。适配可用时，赔付在 35.839、35.888 和 35.864 s 被观察到。驱动程序随后提交迟到的源交易，签发组织的 100 CAL 得到偿还。这 35.8 s 包含 30-s 的义务期、公共时钟的推进、后台调度和轮询，不是密码学计算的时间。历史表示在赔付之后 1.13–1.16 s 完成物化（36.994、37.021 和 37.019 s）。

全部四个适配服务暂停时，赔付在 35.837、35.814 和 35.891 s 被观察到（图 6b）。驱动程序同样提交迟到的源交易，脚本仅在全四个委员会状态都显示义务已偿还、且历史表示既未提交也未物化之后，才解除暂停。随后历史表示完成提交与物化，观察时间为 42.748、42.736 和 42.798 s，其中包含暂停所持续的时间。

对照实验单独隔离出补交这一步，暂停则单独隔离出对适配的依赖。所观察到的先后顺序表明，在 decision-first 版本中，经济关闭与偿还的进行独立于适配。

在全部六个对照与暂停轮次中，签发组织支付 100 CAL 并得到 100 CAL 的偿还，没有新增用户输出，备付金得以恢复。暂停之后的历史表示既不再扣减，也不再记入。24 项委员会观察（六轮、四名成员）确认历史表示已提交并物化、区块体已改变、身份检查成立；完整区块身份由物化检查和共识集成测试验证（见补充材料）。全部九次功能运行都通过审计（四个委员会成员的状态哈希相同、缺口为零、支付已关闭、outbox 已排空），补交与对照两种处理还各自带有三次各 300 笔普通支付的运行，全部完成。

普通路径的开销。六次运行比较位于分支基点（提交 0e7bae5）的版本与最终的 decision-first 快照，顺序为 AB、BA、AB。每次以 100 笔/秒发送 300 笔支付，并发快速请求 64 个，未完成请求 256 个。取各次运行统计量的中位数，接收 P50 在改动前为 1.096 ms、改动后为 1.071 ms，接收 P95 为 27.441 和 30.530 ms，区块观察 P50 为 1,101 和 1,104 ms，包含全部公共确认与成员跟随的完成时间为 3.815 和 3.845 s。各次运行的范围相互重叠：P50 为 1.052–1.112 对 1.045–1.073 ms，P95 为 17.951–39.945 对 22.084–33.744 ms，完成时间为 3.805–3.871 对 3.791–4.000 s，全部 1,800 笔支付均完成。这些是短时输入负载下的普通路径测量。它们既不能证明零开销，也不能给出吞吐上限；而且普通的最终输出不携带输入证书，所以这些运行没有检验大量待决支付所保留证书的存储。

历史表示的批处理。我们在基线代码线上对历史表示命令做了结构性微基准测试（Windows amd64, Go 1.27.1），

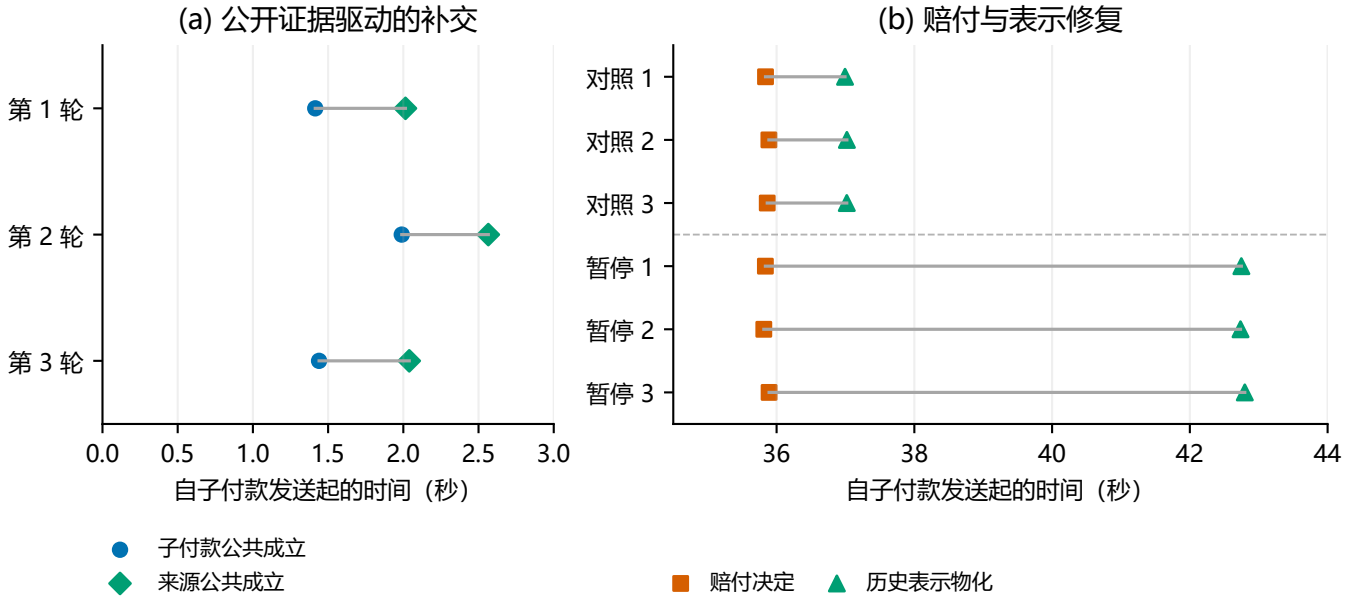


图 6. decision-first 版本中被扣留源交易的解决过程，每种处理三轮。(a) 启用补交时，子支付与被扣留的源交易完成公共执行的观察时间（从子支付的钱包发送算起，0–3 s）。(b) 适配可用与全部四个适配服务暂停两种情形下，赔付与物化（修订区块体的物理安装）的观察时间（34.5–44 s）。两个子图使用不同的横轴范围。所有时间都是包含轮询的客户端观察。偿还以及恢复适配的时刻没有加时间戳；四个委员会状态在适配恢复之前已确认偿还。

把 $k \in \{1, 2, 4, 8\}$ 个缺失输入放在同一个历史区块的同一个分片中。五次重复中，单条命令需要 k 条命令和 k 次分片适配，而一个批处理只需各一次，输入适配仍为 k 次。在 $k = 8$ 时，编码命令总字节数从 127,336 降到 2,752。这些计数来自早期的耦合命令，它同时还承载经济步骤。decision-first 版本为每个已决定的输出增加一条 88 字节的决定命令，上述计数并不包含它，所以它们并不给出当前协议的命令流量总量。另一个分散对照把八个输入放在八个独立的目标区块中，仍需要八条命令和八次分片适配，而字节数从 30,008 降到 5,440，所以紧凑字段解释了一部分下降，共享的区块表示则解释了适配次数的减少。这些是编码后的命令大小，不是网络流量或密码学计算量；补充材料有对应的图和一次归档的批处理集成运行。

一项使用真实门限 RSA 的回归测试，对由 1、2、4 和 8 个输入组成、跨越两个真实分片的一组已决定输入，分别从原始区块出发：作为一个批处理应用；按顺序逐条应用；以及按逆序、以一个、两个和三个为一组应用。每种方式最终得到相同的最终区块体和分片 opening，而命令历史和修订号不同。这是对收敛性质的有限检查，不是网络测量。

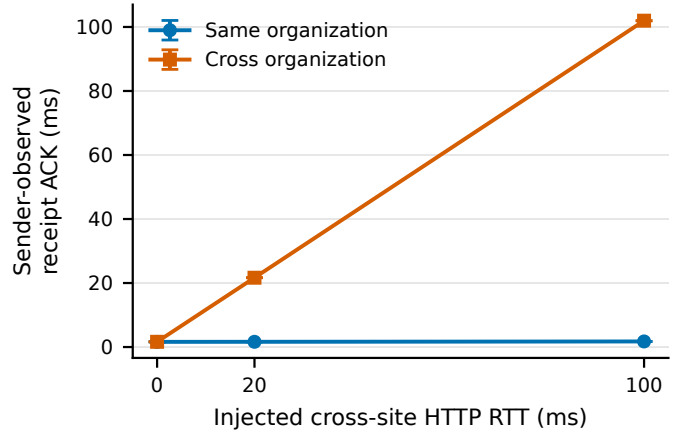


图 7. ACK 延迟 (ms) 与注入的跨站点 HTTP RTT (ms) 的关系。每个点是前 120-s 样本组的三个运行级 P50 的中位数；误差条跨越这些 P50 的最小值与最大值。总提供速率为 100 笔/秒。委员会 P2P 未加延迟。

D. 跨注册组织的续花

为测量通信距离的影响，我们在补充材料所记录的一个归档版本上运行了双组织套件：两个钱包进程各持 32 个身份，经 64 个 lane 发送，每个组织提供 50 笔/秒，24 次运行共完成 342,000 笔支付。延迟比较使用十跳的同组织与跨组织负载，注入的 HTTP RTT 为 0、20 和 100 ms，每个条件重复三次。每个代理方向增加一半的 RTT，组织内部认证和委员会 P2P 不注入延迟。

对共同的前 120-s 样本组，三个运行级跨组 ACK

P50 的中位数依次为 1.632、21.658 和 101.967 ms, 同组数值范围为 1.615–1.744 ms (图 7)。跨组 ACK P50 随注入 RTT 变化, 测得的超出部分为 1.6–2.0 ms。ACK 包含返回路径, 但收款方可以在接受之后即刻发起其后继交易, 无需等待该 ACK 到达前一个发送方。在 100-ms 跨组 RTT 下, 三次 300-s 运行各完成 30,000 笔支付; 待处理峰值为 106–110, 并能排空, 没有持续的尾部增长。该实验在一台物理主机内隔离出 HTTP 路径延迟。

E. 运行容量与服务可用性

独立输入吞吐 (基线版本)。三次各 30,000 笔支付的运行以 2,000 笔/秒为目标, 使用九服务的内存配置、NoSync 钱包和经授权的组织代付 FUEL。成员进程使用 200% 的 Go GC 目标和 16 个运行时处理器。全部支付完成, 四个委员会状态一致。含排空的完成速率为 1,750.27、1,820.91 和 1,887.31 笔/秒, 历时 15.9–17.1 s, 接收 P50 为 2.646、2.842 和 2.824 ms, P95 为 47.9、49.1 和 48.0 ms。驱动程序记录到有 6,551–7,570 笔支付需要等待快速接收准入名额, P95 派发滞后为 179–229 ms, 所以提供的负载并不是均匀的 2,000 笔/秒。这些是基线版本上的短时完成测量, 没有在 decision-first 版本上重新测量; 该版本在普通支付上的开销由上文的六次运行对比检验。补充材料说明了 150,000 笔支付系列所用的版本和时长。

在归档版本上, 冗余认证与已 INSTALL 的证据在所述故障下维持服务。九次每秒 200 笔的运行完成 216,000 笔支付, 其间有一个成员被延迟或暂停 30 s。网关丢失对照中, 一名成员持有经 INSTALL 复制的完整材料, 备用提交在 INSTALL 之后 2.503–2.508 s 出现。冲突对照既没有出现相互冲突的双 QC, 也没有重复的本金或费用效果, 而部分批准仍然是一种额度代价。

补充材料第 S2 节的多身份测量报告每笔支付的 CPU 和 HTTP 负载开销; 2,048 个身份共享客户端。历史内存增长与积压分别报告。该节还给出 LND regtest 参照, 其完成事件、持久化设置和负载并发度与 Next 的测量不同。

VII. 结论

Next 让收款方能够依据已认证的证据继续花费, 同时由一个明确的、有备付金支撑的签发组织对每一个缺失的源交易负责。在基线版本上, 100 跳链完成最后一次认证接收的用时中位数为 158.846 ms, 而在各跳之间

等待公共确认则需要 65.204 s; 九轮实验关闭了 21,600 笔支付, 并偿还了全部 216 次赔付。另外两项性质规定了缺失来源如何关闭。第一, 延迟只改变备付金的临时占用: 当所有源交易都执行时, 到达与赔付的任意顺序最终得到相同的最终持有者。第二, 赔付决定是一条只需要委员会的公共记录; 随后把该扣减记录进已存储区块的历史表示路径可以滞后或停止, 而不妨碍义务关闭。decision-first 实验印证了这一点: 在全部四个适配服务暂停时, 赔付与偿还仍然完成, 并且被扣留的源交易通过原签署者的补交得以执行。这些路径将支付续花与来源交付分离, 并将经济关闭与历史物化分离。

致谢

ChatGPT 与 Claude 辅助起草和修订摘要、正文及中文阅读版; Codex 辅助基于源码的审阅、修订、LaTeX 组装与绘图脚本。实验图表由所记录的测量数据生成。

参考文献

- [1] M. Baudet, G. Danezis, and A. Sonnino, “FastPay: High-Performance Byzantine Fault Tolerant Settlement,” in *Proceedings of the ACM Conference on Advances in Financial Technologies*, 2020, pp. 163–177.
- [2] M. Baudet, A. Sonnino, M. Kelkar, and G. Danezis, “Zef: Low-latency, Scalable, Private Payments,” in *Proceedings of the Workshop on Privacy in the Electronic Society*, 2023, pp. 1–16.
- [3] S. Blackshear, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, X. Li, M. Logan, A. Menon, T. Nowacki, A. Sonnino, B. Williams, and L. Zhang, “Sui Lutris: A Blockchain Combining Broadcast and Consensus,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 2606–2620.
- [4] L. Aumayr, P. Moreno-Sanchez, A. Kate, and M. Maffei, “Blitz: Secure Multi-Hop Payments Without Two-Phase Commits,” in *30th USENIX Security Symposium*, 2021, pp. 4043–4060. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/aumayr>
- [5] —, “Breaking and Fixing Virtual Channels: Domino Attack and Donner,” in *Network and Distributed System Security Symposium*, 2023.
- [6] S. Sridhar, A. Sonnino, and L. Kokoris-Kogias, “Stingray: Fast Concurrent Transactions Without Consensus,” 2025, arXiv:2501.06531, version 1. [Online]. Available: <https://arxiv.org/abs/2501.06531>
- [7] S. Mazumdar and S. Ruj, “CryptoMaze: Privacy-Preserving Splitting of Off-Chain Payments,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 2, pp. 1060–1073, 2023.
- [8] V. Mavroudis, K. Wüst, A. Dhar, K. Kostianen, and S. Čapkun, “Snappy: Fast On-Chain Payments with Practical Collaterals,” in *Network and Distributed System Security Symposium*, 2020. [Online]. Available: <https://arxiv.org/abs/2001.01278>
- [9] A. Madhusudan, M. Sedaghat, S. Tiwari, K. Cong, and B. Preneel, “Reusable, instant and private payment guarantees for cryptocurrencies,” in *Information Security and Privacy (ACISP)*, 2023, pp. 580–605.
- [10] Z. Liu, Y. Xiang, J. Shi, P. Gao, H. Wang, X. Xiao, B. Wen, Q. Li, and Y.-C. Hu, “Make Web3.0 Connected,” *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 2965–2981, 2022.

- [11] G. Ateniese, B. Magri, D. Venturi, and E. Andrade, “Redactable Blockchain—or—Rewriting History in Bitcoin and Friends,” in *IEEE European Symposium on Security and Privacy*, 2017, pp. 111–126. [Online]. Available: <https://eprint.iacr.org/2016/757>
- [12] D. Zhang, J. Le, X. Lei, T. Xiang, and X. Liao, “Secure Redactable Blockchain With Dynamic Support,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 2, pp. 717–731, 2024.
- [13] X. Li, J. Xu, L. Yin, Y. Lu, Q. Tang, and Z. Zhang, “Escaping From Consensus: Instantly Redactable Blockchain Protocols in Permissionless Setting,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 5, pp. 3699–3715, 2023.
- [14] K. Y. Chan, L. Chen, Y. Tian, and T. H. Yuen, “Reconstructing Chameleon Hash: Full Security and the Multi-Party Setting,” in *Proceedings of the ACM Asia Conference on Computer and Communications Security*, 2024, pp. 1076–1091.
- [15] S. A. K. Thyagarajan, A. Bhat, B. Magri, D. Tschudi, and A. Kate, “Reparo: Publicly Verifiable Layer to Repair Blockchains,” in *Financial Cryptography and Data Security*, 2021, pp. 37–56. [Online]. Available: <https://arxiv.org/abs/2001.00486>
- [16] R. Matzutt, V. Ahlrichs, J. Pennekamp, R. Karwacik, and K. Wehrle, “A Moderation Framework for the Swift and Transparent Removal of Illicit Blockchain Content,” in *IEEE International Conference on Blockchain and Cryptocurrency*, 2022.
- [17] X. Chen and Y. Gao, “CDEdit: A Highly Applicable Redactable Blockchain with Controllable Editing Privilege and Diversified Editing Types,” 2022, arXiv:2205.07054. [Online]. Available: <https://arxiv.org/abs/2205.07054>
- [18] E. Buchman, J. Kwon, and Z. Milosevic, “The latest gossip on BFT consensus,” 2019, arXiv:1807.04938, version 3. [Online]. Available: <https://arxiv.org/abs/1807.04938v3>